



PyTorch

Agenda

- PyTorch Basic
- Machine Learning using PyTorch
- Deep learning using PyTorch
- NLP using PyTorch
- Advanced
- Modern Network Architectures
- Computer Vision

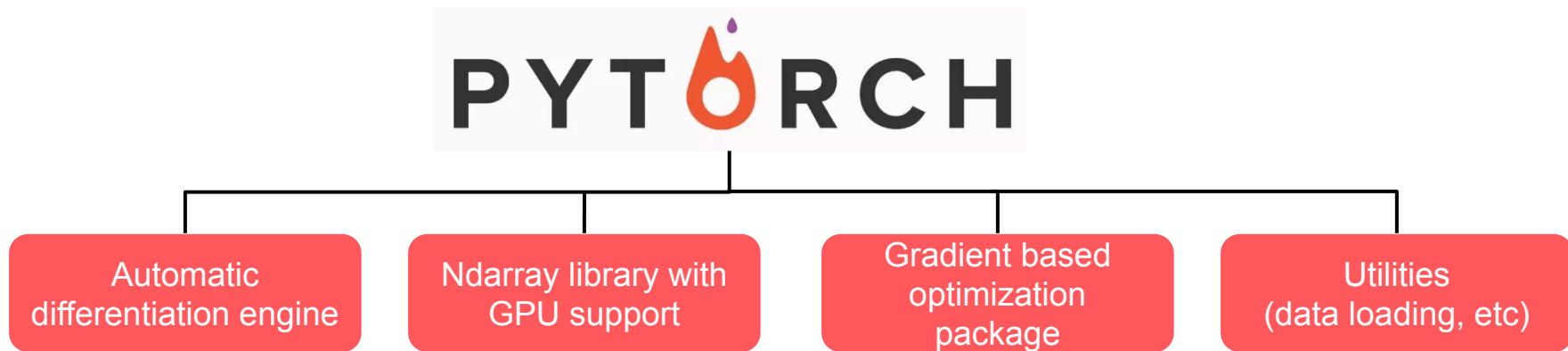


PyTorch Basic

What is PyTorch?

It is a Python-based scientific computing package targeted at two sets of audiences:

- A replacement for NumPy to use the power for GPUs
- A deep learning research platform that provides maximum flexibility and speed



Advantages of PyTorch

- Simple Interface: It offers easy to use API, thus it is very simple to operate and run like Python.
- Pythonic in nature: This library, being Pythonic, smoothly integrates with the python data science stack. Thus it can leverage all the services and functionalities offered by the Python environment.
- Computational graphs: In addition to this, PyTorch provides an excellent platform which offers dynamic computational graphs, thus you can change them during runtime. This is highly useful when you have no idea how much memory will be required for creating a network model.

PyTorch: Three Levels of Abstraction

- Tensor: Like array in Numpy, but runs on GPU
- Variable: Node in a computational graph; stores data and gradient
- Module: A neural network layer; may store state or learnable weights

Tensor

- PyTorch Tensors are just like numpy arrays, but they can run on GPU.
- No built-in notion of computational graph, or gradients, or deep learning.
- Here we fit a two-layer net using PyTorch Tensors
- `torch.Tensor` is an alias for the default tensor type (`torch.FloatTensor`).
- A tensor can be constructed from a Python list or sequence using the `torch.tensor()` constructor:

```
import torch
import numpy as np

torch.tensor([[1., -1.], [1., -1.]])

tensor([[ 1., -1.],
        [ 1., -1.]])

torch.tensor(np.array([[1, 2, 3], [4, 5, 6]]))

tensor([[1, 2, 3],
        [4, 5, 6]])
```

Operations

- The syntax on a tensor operation

```
import torch

# Initialize

x = torch.Tensor(2, 3) # An un-initialized Tensor object.x holds garbage data.
y = torch.rand(2, 3)   # Initialize with random values

# Operations

z1 = x + y
z2 = torch.add(x, y)    # Same as above

print(z2)

tensor([[0.5894, 0.1947, 0.4298],
        [0.1288, nan, 0.8964]])
```

Variable

- A Variable wraps a Tensor.
- It supports nearly all the API's defined by a Tensor.
- Variable also provides a *backward* method to perform backpropagation.
- For example:
- To backpropagate a loss function to train model parameter x , we can use a variable *loss* to store the value computed by a loss function. Then, we call *loss.backward* which computes the gradients $\partial \text{loss} / \partial x$ for all trainable parameters.
- PyTorch will store the gradient results back in the corresponding variable.

Variable

This is the syntax for declaring variable.

```
import torch
from torch.autograd import Variable
# Variables wrap a Tensor
x = Variable(torch.ones(2, 2), requires_grad=True)
# Variable containing:
# 1  1
# 1  1
# [torch.FloatTensor of size 2x2]
#requires_grad indicates whether a variable is trainable.
y = x + 2
# z = torch.add(x,y)
print(y)
```

Output:
tensor([[3., 3.],
 [3., 3.]])

Autograd

- The autograd package provides automatic differentiation for all operations on Tensors.
- It performs the backpropagation starting from a variable.
- In deep learning, this variable often holds the value of the cost function.
- *Backward* executes the backward pass and computes all the backpropagation gradients automatically.
- We access individual gradient through the attributes *grad* of a variable. *x.grad* below returns a 2x2 gradient tensor for $\partial \text{out} / \partial x$.

Autograd

- Importing torch library
- Create a tensor
- Requires_grad = True is to track computation with the tensor
- backward() is doing backpropagation to compute gradient

```
import torch
# Creating the graph
x = torch.tensor(1.0, requires_grad = True)
z = x ** 3
z.backward() #Computes the gradient
print(x.grad.data)
```

tensor(3.)

Dynamic Graphics

- PyTorch uses a dynamic computation graph.
- Whenever we create a variable or operations, it is executed immediately.
- We can add and execute operations anytime before *backward* is called.
- *Backwards* follows the graph backward to compute the gradients. Then the graph will be disposed.

Dynamic Graphics

- Advantages:
 - The main utility of a dynamic computation graph is that it allows you to process complex inputs and outputs, without worrying to convert every batch of input into a big fat tensor.
 - A major utility of this comes while running recurrent neural networks on variable length inputs and outputs.

Models in PyTorch

- `Torch.nn.Module` is base class for all neural network modules.
- Your models should also subclass this class.
- Modules can also contain other Modules, allowing to nest them in a tree structure.

You can assign the submodules as regular attributes.

- To create a Module , one has to inherit from the base class and implement the constructor `init (self, ...)` and the forward pass `forward(self, x)`.

Models in PyTorch

- Create a class named Net
- nn.Module is passed to the class as a parameter
- This class is inherited from nn.Module
- Convolution parameters are initialized
- Fully connected layer parameters are initialized
- The forward function contains max pooling , relu activation function and fully connected layer.

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):

    def __init__( self):
        super (Net,self). __init__()
        self.conv1=nn.Conv2d(1,32,kernel_size =5)
        self.conv2=nn.Conv2d(32,64,kernel_size =5)
        self.fc1=nn.Linear(256,200)
        self.fc2=nn.Linear(200,10)

    def forward ( self , x):
        x=F.relu(F.max_pool2d(self.conv1(x),kernel_size =3,stride =3))
        x=F.relu(F.max_pool2d(self.conv2(x),kernel_size =2,stride =2))
        x=x.view(-1,256)
        x=F.relu(self.fc1(x))
        x=self.fc2(x)
        return x
```

Models in PyTorch

- As long as you use autograd-compliant operations, the backward pass is implemented automatically.
- Modules added as attributes are seen by `Module.parameters()` , which returns an iterator over the model's parameters for optimization.
- `__init__` method initialize parameters of models
- `Module.parameters()`
Contains all parameter which are initialize parameter
- Parameters added as attributes are also seen by `Module.parameters()`.

```
class Net(nn.Module):

    def __init__(self):
        super(Net,self).__init__()
        self.conv1=nn.Conv2d(1,32,kernel_size=5)
        self.conv2=nn.Conv2d(32,64,kernel_size=5)
        self.fc1=nn.Linear(256,200)
        self.fc2=nn.Linear(200,10)

model=Net()
for k in model.parameters():
    print (k.size())
```

TensorBoard in PyTorch

With PyTorch 1.1.0, TensorBoard is now natively supported in PyTorch. TensorBoard is a visualization library for TensorFlow that is useful in understanding training runs, tensors, and graphs. There have been 3rd-party ports such as tensorboardX but no official support until now.

The following three install commands will install PyTorch 1.1.0 with Tensorboard 1.14.

- `pip install --upgrade torch`
- `pip install tb-nightly`
- `pip install future`

Running Tensorboard

```
pip install --upgrade torchvision
```

- [illegible]

TensorBoard in PyTorch

- Data loader loads the dataset from dataset object
- Create ResNet model
- Create grid object to visualize images in grid
- Writer writes output to graph

```
# Data loader loads the dataset from datasets object
trainloader = torch.utils.data.DataLoader(trainset, \
                                           batch_size=64, shuffle=True)

# Load ResNet model from models module
model = torchvision.models.resnet50(False)
# Have ResNet model take in grayscale rather than RGB
model.conv1 = torch.nn.Conv2d(1, 64, kernel_size=7, \
                              stride=2, padding=3, bias=False)

# Iterate each image one by one
images, labels = next(iter(trainloader))

# Create grid in background
grid = torchvision.utils.make_grid(images)
writer.add_image('images', grid, 0)
writer.add_graph(model, images)
writer.close()
```

- The PyTorch module `torch.optim` provides many optimizers.
- An optimizer has an internal state to keep quantities such as moving averages, and operates on an iterator over parameters.
 - Values specific to the optimizer can be specified to its constructor, and
 - Its `step` method updates the internal state according to the `grad` attributes of the `Parameters`, and updates the latter according to the internal state.

Optimizer

- Example:
 - `torch.optim.SGD` (momentum, and Nesterov's algorithm),
 - `torch.optim.Adam`
 - `torch.optim.Adadelta`
 - `torch.optim.Adagrad`
 - `torch.optim.RMSprop`
 - `torch.optim.LBFGS`

Stochastic Gradient Descent:

- The word '*stochastic*' means a system or a process that is linked with a random probability.
- Hence, in Stochastic Gradient Descent, a few samples are selected randomly instead of the whole data set for each iteration.

torch.optim.SGD

The syntax is for SGD in pytorch.

torch.optim.SGD(params, lr=0.1, momentum=0, weight_decay=0, dampening = 0, nesterov=False)

- **params** (*iterable*) — iterable of parameters to optimize or dicts defining parameter groups
- **lr** (*float*) — learning rate
- **momentum** (*float, optional*) — momentum factor (default: 0)
- **weight_decay** (*float, optional*) — weight decay (L2 penalty) (default: 0)
- **dampening** (*float, optional*) — dampening for momentum (default: 0)
- **nesterov** (*bool, optional*) — enables Nesterov momentum (default: False)

torch.optim.Adam

torch.optim.Adam(*params*, *lr*=0.001, *betas*=(0.9, 0.999), *eps*=1e-08, *weight_decay*=0, *amsgrad*=False)

- **params** (*iterable*) — iterable of parameters to optimize or dicts defining parameter groups
- **lr** (*float, optional*) — learning rate (default: 1e-3)
- **betas** (*Tuple[float, float], optional*) — coefficients used for computing running averages of gradient and its square (default: (0.9, 0.999))
- **eps** (*float, optional*) — term added to the denominator to improve numerical stability (default: 1e-8)
- **weight_decay** (*float, optional*) — weight decay (L2 penalty) (default: 0)
- **amsgrad** (*boolean, optional*) — whether to use the AMSGrad variant of this algorithm from the paper On the Convergence of Adam and Beyond (default: False)

Loss Function

- A loss function takes the (output, target) pair of inputs, and computes a value that estimates how far away the output is from the target.
- There are several different loss functions under the nn package.

Loss Function

- For example:
 - `binary_cross_entropy`
 - `binary_cross_entropy_with_logits`
 - `poisson_nll_loss`
 - `cosine_embedding_loss`
 - `cross_entropy`
 - `ctc_loss`
 - `hinge_embedding_loss`
 - `kl_div`
 - `L1_loss`
 - `mse_loss`

Binary_cross_entropy

- Function that measures the Binary Cross Entropy between the target and the output.
`torch.nn.functional.binary_cross_entropy(input, target, weight=None, size_average=None, reduce=None, reduction='mean')`
- Parameters:
 - input – Tensor of arbitrary shape
 - target – Tensor of the same shape as input
 - weight (*Tensor, optional*) – a manual rescaling weight if provided it's repeated to match input tensor shape
 - size_average (*bool, optional*) – Deprecated (see `reduction`).
 - reduce (*bool, optional*) – Deprecated (see `reduction`)
 - reduction (*string, optional*) – Specifies the reduction to apply to the output: `'none'` | `'mean'` | `'sum'`.

Binary_cross_entropy_with_logits

- Function that measures Binary Cross Entropy between target and output logits.
`torch.nn.functional.binary_cross_entropy_with_logits(input, target, weight=None, size_average=None, reduce=None, reduction='mean', pos_weight=None)`
- Parameters:
 - input – Tensor of arbitrary shape
 - target – Tensor of the same shape as input
 - weight (*Tensor, optional*) – a manual rescaling weight if provided it's repeated to match input tensor shape
 - size_average (*bool, optional*) – Deprecated (see reduction).
 - reduce (*bool, optional*) – Deprecated (see reduction)
 - reduction (*string, optional*) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'.
 - pos_weight (*Tensor, optional*) – a weight of positive examples. Must be a vector with length equal to the number of classes.

mse_loss

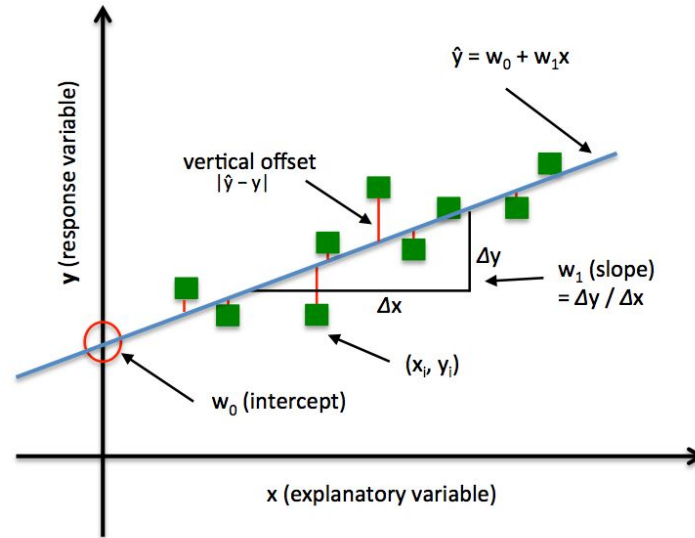
- Measures the element-wise mean squared error.
`torch.nn.MSELoss(size_average=None, reduce=None, reduction='mean')`
- Parameters:
 - `size_average` (*bool, optional*) – By default, the losses are averaged over each loss element in the batch.
 - `reduce` (*bool, optional*) – By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`
 - `reduction` (*string, optional*) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'.



Machine Learning using PyTorch

Linear Regression

- Linear Regression is an approach that tries to find a linear relationship between the response variable and an explanatory variable by minimizing the distance as shown below.



Linear Regression

- Let's see how to implement linear regression in pytorch

- Importing library

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
```

- Initializing hyper-parameters

```
# Hyper-parameters
input_size = 1
output_size = 1
num_epochs = 60
learning_rate = 0.001
```

- Create dummy dataset

```
# Toy dataset
x_train = np.array([[3.3], [4.4], [5.5], [6.71], [6.93], [4.168],
                    [9.779], [6.182], [7.59], [2.167], [7.042],
                    [10.791], [5.313], [7.997], [3.1]], dtype=np.float32)

y_train = np.array([[1.7], [2.76], [2.09], [3.19], [1.694], [1.573],
                    [3.366], [2.596], [2.53], [1.221], [2.827],
                    [3.465], [1.65], [2.904], [1.3]], dtype=np.float32)
```

Linear Regression

- Create model object
- Initializing optimizer and loss function

```
# Linear regression model
model = nn.Linear(input_size, output_size)

# Loss and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

- [illegible]

Linear Regression

- Plot the original data
- The `detach()` method constructs a new view on a tensor which is declared not to need gradients, i.e., it is to be excluded from further tracking of operations, and therefore the subgraph involving this view is not recorded.

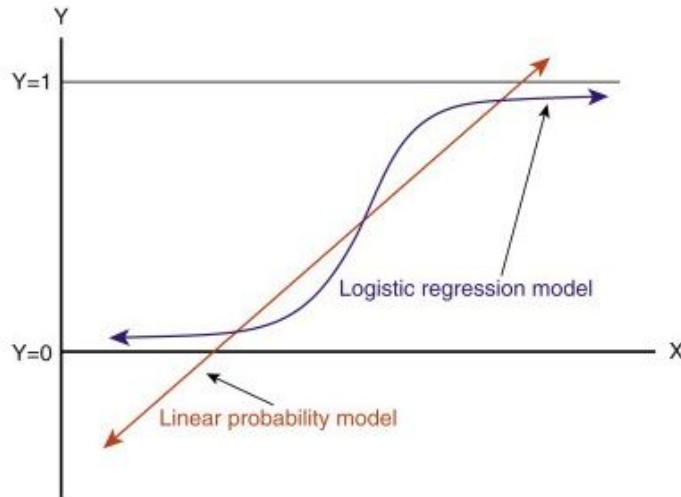
```

# Plot the graph
predicted = model(torch.from_numpy(x_train)).detach().numpy()
plt.plot(x_train, y_train, 'ro', label='Original data')
plt.plot(x_train, predicted, label='Fitted line')
plt.legend()
plt.show()

# Save the model checkpoint
torch.save(model.state_dict(), 'model.ckpt')
  
```

Logistic Regression

- **Logistic regression** is used to describe data and to explain the relationship between **one dependent binary variable** and one or more nominal, ordinal, interval or ratio-level independent variables.
- The figure below shows the difference between **Logistic** and **Linear** regression.



- Importing dataset from torchvision.datasets module

[illegible]

Logistic Regression

- The following code loads data and converts from numpy format of the dataset to tensor.

Logistic Regression

- Create model object and gets input size of dataset and number of classes
- Create object to calculate loss
- Create optimizer object

```
# Logistic regression model
model = nn.Linear(input_size, num_classes)

# Loss and optimizer
# nn.CrossEntropyLoss() computes softmax internally
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Logistic Regression

- Reshape image and feed the images to model
- In the forward pass, we have to calculate loss
- In the backward pass, we don't want any gradient from previous epoch, so we are using `zero_grad()`

```
# Train the model
total_step = len(train_loader)
for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_loader):
        # Reshape images to (batch_size, input_size)
        images = images.reshape(-1, 28*28)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if (i+1) % 100 == 0:
        print ('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'
              .format(epoch+1, num_epochs, i+1, total_step, loss.item()))
```

Logistic Regression

- no_grad() does not calculate gradients
- Reshape image and feed the images to model
- Saving the model named model.ckpt

```
# Test the model
# In test phase, we don't need to compute gradients (for memory efficiency)
with torch.no_grad():
    correct = 0
    total = 0
    for images, labels in test_loader:
        images = images.reshape(-1, 28*28)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum()

    print('Accuracy of the model on the 10000 test images: {} %'\
          .format(100 * correct / total))

# Save the model checkpoint
torch.save(model.state_dict(), 'model.ckpt')
```

Feedforward Neural Network

- The feedforward models have hidden layers in between the input and the output layers.
- After every hidden layer, an activation function is applied to introduce non-linearity.

Feedforward Neural Network

- nn.Module is passed to the class as a parameter
- This class is inherited from nn.Module
- Linear layer is initialized with input size and hidden size
- The forward function contains linear layer, relu activation function and fully connected layer.

```
class Feedforward(torch.nn.Module):
    def __init__(self, input_size, hidden_size):
        super(Feedforward, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.fc1 = torch.nn.Linear(self.input_size, self.hidden_size)
        self.relu = torch.nn.ReLU()
        self.fc2 = torch.nn.Linear(self.hidden_size, 1)
        self.sigmoid = torch.nn.Sigmoid()

    def forward(self, x):
        hidden = self.fc1(x)
        relu = self.relu(hidden)
        output = self.fc2(relu)
        output = self.sigmoid(output)
        return output
```

- Example: To implement feedforward neural network in pytorch, we take a dataset from sklearn.dataset with 2 input features and output.

Feedforward Neural Network

- Importing libraries
- Import make blobs data points from sklearn.datasets
- Get training dataset with 2 input features and output
- Converting the data points from numpy to tensor
- Get testing data points with 2 input features and output.
- Converting of test data points from numpy to tensor

```
# Importing libraries
import torch
import numpy
# Create random datapoint
from sklearn.datasets import make_blobs
def blob_label(y, label, loc): # assign labels
    target = numpy.copy(y)
    for l in loc:
        target[y == l] = label
    return target
# Get input and output dataset for training
x_train, y_train = make_blobs(n_samples=40, n_features=2, \
                             cluster_std=1.5, shuffle=True)
# Converting from numpy datapoints to float tensor
x_train = torch.FloatTensor(x_train)
y_train = torch.FloatTensor(blob_label(y_train, 0, [0]))
y_train = torch.FloatTensor(blob_label(y_train, 1, [1,2,3]))
# Get input and output dataset for testing
x_test, y_test = make_blobs(n_samples=10, n_features=2, \
                             cluster_std=1.5, shuffle=True)
# Converting from numpy datapoints to float tensor
x_test = torch.FloatTensor(x_test)
y_test = torch.FloatTensor(blob_label(y_test, 0, [0]))
y_test = torch.FloatTensor(blob_label(y_test, 1, [1,2,3]))
```

Feedforward Neural Network

- nn.Module is passed to the class as a parameter
- This class is inherited from nn.Module
- Linear layer is initialized with input size and hidden size
- The forward function contains linear layer, relu activation function and fully connected layer.

```
# Creating a feedforward neural network
# Initializing neural network parameters
# Creating layers in feedforward neural network
class Feedforward(torch.nn.Module):
    def __init__(self, input_size, hidden_size):
        super(Feedforward, self).__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.fc1 = torch.nn.Linear(self.input_size, self.hidden_size)
        self.relu = torch.nn.ReLU()
        self.fc2 = torch.nn.Linear(self.hidden_size, 1)
        self.sigmoid = torch.nn.Sigmoid()
    def forward(self, x):
        hidden = self.fc1(x)
        relu = self.relu(hidden)
        output = self.fc2(relu)
        output = self.sigmoid(output)
        return output
```

Feedforward Neural Network

- Create a model object
- BCE Loss is Binary cross entropy to calculate loss
- SGD is used to optimize weights and it takes first model.parameters() as parameter and learning

```
# Creating a model object
# Creating a object for loss function
# Creating a object for optimizer
model = Feedforward(2, 10)
criterion = torch.nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr = 0.01)
```

Feedforward Neural Network

- The following codes measure the performance of a model before the model trained.

```
# Evaluating the model performance before train the model
model.eval()
y_pred = model(x_test)
before_train = criterion(y_pred.squeeze(), y_test)
print('Test loss before training' , before_train.item())
```

Test loss before training 0.9190487861633301

Feedforward Neural Network

- Train the model Epoch size is initialized
- Zero_grad method make the gradient value zero when the mode moves to next epoch

```
# Training the model
model.train()
epoch = 20
for epoch in range(epoch):
    optimizer.zero_grad()
    # Forward pass
    y_pred = model(x_train)
    # Compute Loss
    loss = criterion(y_pred.squeeze(), y_train)

    print('Epoch {}: train loss: {}'.format(epoch, loss.item()))
    # Backward pass
    loss.backward()
    optimizer.step()
```

Feedforward Neural Network

- The following codes measure the performance of a model after the model trained.

```
# Evaluate the model performance after the train of model.
model.eval()
y_pred = model(x_test)
after_train = criterion(y_pred.squeeze(), y_test)
print('Test loss after Training' , after_train.item())
```

Test loss after Training 2.4854137897491455

Convolutional Neural Network

- CNNs are a type of deep layer neural networks, used to learn Filters that when convolved with the image, can be used to extract features. An example:

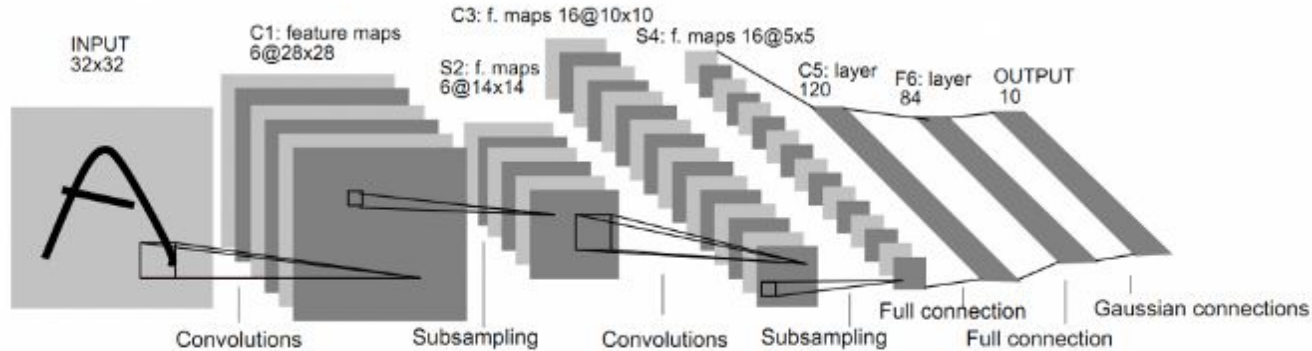


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Convolutional Neural Network

Training an image classifier

We will do the following steps in order:

- Load and normalizing the CIFAR10 training and test datasets using torchvision
- Define a Convolutional Neural Network
- Define a loss function
- Train the network on the training data
- Test the network on the test data

Convolutional Neural Network

- We will use the CIFAR10 dataset. It has the classes: 'airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck'. The images in CIFAR-10 are of size 3x32x32, i.e. 3-channel color images of 32x32 pixels in size.
- Build the CNN to classify images

```
# Importing libraries
import torch
import torchvision
import torchvision.transforms as transforms
```

Convolutional Neural Network

- Normalize does the following for each channel: $\text{image} = (\text{image} - \text{mean}) / \text{std}$
The parameters mean, std are passed as 0.5. This will normalize the image in the range $[-1, 1]$. For example, the minimum value 0 will be converted to $(0-0.5)/0.5=-1$, the maximum value of 1 will be converted to $(1-0.5)/0.5=1$.
- if we would like to get our image back in $[0, 1]$ range, we could use,
- $\text{image} = ((\text{image} * \text{std}) + \text{mean})$

```
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

# Downloading the CIFAR dataset for training
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)

# Loading the dataset
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                           shuffle=True, num_workers=2)

# Downloading the CIFAR dataset for testing
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                         shuffle=False, num_workers=2)

# Labels name
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

Convolutional Neural Network

- Visualize some images
- Create a function to visualize images
- Get some random training Images
- Torchvision.utils.make_grid get tensor value of images and call imshow function to visualization.

```
# Visualizing some images
import matplotlib.pyplot as plt
import numpy as np

# functions to show an image
def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# get some random training images
dataiter = iter(trainloader)
images, labels = dataiter.next()

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join('%5s' % classes[labels[j]] for j in range(4)))
```

Convolutional Neural Network

- Create Net class to form cnn network. This class is a subclass for nn.Module.
- Initializing cnn layers with input 3 dimension, output 6 dimension and kernel size is 5.
- MaxPool2d is also initializing following the cnn layer
- Three fully connected layers are also initialized
- The forward function is forward propagation. It contains network layer with activation function.
- View function is for reshape the tensor

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```

Convolutional Neural Network

- The following codes contains loss function and optimizer.

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

Convolutional Neural Network

- Train the model
- Get input data
- Gradients value is zero.
- Forward propagation
- Calculate loss
- Backward propagation
- Print loss every 2000 mini-batches

```
for epoch in range(2): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999: # print every 2000 mini-batches
            print('[%d, %5d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0

    print('Finished Training')
```

Convolutional Neural Network

- Visualizing some image

```
dataiter = iter(testloader)
images, labels = dataiter.next()

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))
```



Convolutional Neural Network

- We will see how much correctly the model is going to predict those images.

```
outputs = net(images)
_, predicted = torch.max(outputs, 1)

print('Predicted: ', ' '.join('%5s' % classes[predicted[j]]
                                for j in range(4)))
```

- Output:

Predicted: cat ship ship ship

Convolutional Neural Network

➤ Let us look at how the network performs on the whole dataset.

➤ When we test the model, we do not need to calculate gradients

```
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print('Accuracy of the network on the 10000 test images: %d %%' % (
    100 * correct / total))
```

Accuracy of the network on the 10000 test images: 53 %

Convolutional Neural Network

- What are the classes that performed well, and the classes that did not perform well

```
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))
with torch.no_grad():
    for data in testloader:
        images, labels = data
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        c = (predicted == labels).squeeze()
        for i in range(4):
            label = labels[i]
            class_correct[label] += c[i].item()
            class_total[label] += 1

for i in range(10):
    print('Accuracy of %5s : %2d %%' % (
        classes[i], 100 * class_correct[i] / class_total[i]))
```

Convolutional Neural Network

- The result shows our model performance
- For example:
The model predicted 57% plane correctly

Accuracy of plane	: 57 %
Accuracy of car	: 72 %
Accuracy of bird	: 44 %
Accuracy of cat	: 14 %
Accuracy of deer	: 31 %
Accuracy of dog	: 48 %
Accuracy of frog	: 78 %
Accuracy of horse	: 60 %
Accuracy of ship	: 60 %
Accuracy of truck	: 62 %

PyTorch Model inference using ONNX

- ONNX is an open format to represent deep learning models.
- With ONNX, AI developers can more easily move models between state-of-the-art tools and choose the combination that is best for them
- ONNX is developed and supported by a community of partners

PyTorch Model inference using ONNX

Framework Interoperability

- Enabling interoperability makes it possible to get great ideas into production faster
- ONNX enables models to be trained in one framework and transferred to another for inference.
- ONNX models are currently supported in caffe2, Microsoft Cognitive Toolkit, MXNet, and PyTorch, and there are connectors for many other common frameworks and libraries

PyTorch Model inference using ONNX

Limitation of ONNX

- The ONNX exporter is a *trace-based* exporter, which means that it operates by executing your model once, and exporting the operators which were actually run during this run. This means that if your model is dynamic, e.g., changes behavior depending on input data, the export won't be accurate.
- Similarly, a trace is likely to be valid only for a specific input size (which is one reason why we require explicit inputs on tracing.)
- Examining the model trace and making sure the traced operators look reasonable.
- PyTorch and Caffe2 often have implementations of operators with some numeric differences. Depending on model structure, these differences may be negligible, but they can also cause major divergences in behavior (especially on untrained models.)

PyTorch Model inference using ONNX

➤ AlexNet from PyTorch to Caffe2:

➤ Importing torch and torch models

➤ Import alexnet model with Pretrained = True parameter

➤ Create sample input

➤ Create label to the input and output layers

➤ torch.onnx.export get model, input, model name in onnx and label names

```
import torch
import torchvision.models as models

# Use an existing model from Torchvision, note it
# will download this if not already on your computer (might take time)
model = models.alexnet(pretrained=True)

# Create some sample input in the shape this model expects
dummy_input = torch.randn(10, 3, 224, 224)

# It's optional to label the input and output layers
input_names = [ "actual_input_1" ] + [ "learned_%d" % i for i in range(16) ]
output_names = [ "output1" ]

# Use the exporter from torch to convert to onnx
# model (that has the weights and net arch)
torch.onnx.export(model, dummy_input, "alexnet.onnx",
                  verbose=True, input_names=input_names,
                  output_names=output_names)
```

PyTorch Model inference using ONNX

➤ Load ONNX model

```
import onnx

# Load the ONNX model
model = onnx.load(modelName)

# Check that the IR is well formed
onnx.checker.check_model(model)

# Print a human readable representation of the graph
onnx.helper.printable_graph(model.graph)
```

PyTorch Model inference using ONNX

➤ Use caffe2 in the backend

```
# ...continuing from above
import caffe2.python.onnx.backend as backend
import numpy as np

rep = backend.prepare(model, device="CUDA:0") # or "CPU"
# For the Caffe2 backend:
#     rep.predict_net is the Caffe2 protobuf for the network
#     rep.workspace is the Caffe2 workspace for the network
#     (see the class caffe2.python.onnx.backend.Workspace)
outputs = rep.run(np.random.randn(10, 3, 224, 224).astype(np.float32))
# To run networks with more than one input, pass a tuple
# rather than a single numpy ndarray.
print(outputs[0])
```

PyTorch Model inference using ONNX

➤ Use caffe2 in the backend

```
# ...continuing from above
import caffe2.python.onnx.backend as backend
import numpy as np

rep = backend.prepare(model, device="CUDA:0") # or "CPU"
# For the Caffe2 backend:
#     rep.predict_net is the Caffe2 protobuf for the network
#     rep.workspace is the Caffe2 workspace for the network
#     (see the class caffe2.python.onnx.backend.Workspace)
outputs = rep.run(np.random.randn(10, 3, 224, 224).astype(np.float32))
# To run networks with more than one input, pass a tuple
# rather than a single numpy ndarray.
print(outputs[0])
```



